@ 12th - october - 2026 - Monday:- built in

I

* Common escape characters :- in print() function :-

Eg: print("Hello World \n")

\n - start a new line
\" - prints a double quote
\t - adds a tab indent
\\ - prints a backslash

↓

escape sequences. () -> Parentheses.

* The input() function always returns the value in form of string as during taking data from console it treats it as an string.

* Python can add texts & texts, values (int) to int, but not both (texts & values).

* Build dynamic messages : (concatenation with +) :-

* Manipulate strings :

-> Concatenation : The basic way to combine text is gluing strings together with the plus + sign. It's the process called concatenation.

Eg:

variable

first = "Hello" -> strings
second = "World"
message = first + " " + last + "!"
print(message)

" " -> 1st gluing
last -> second gluing
"!" -> third gluing

Concatenate, format, & calculate: (modern way of concatenation):-

→ is f-string:- (formatted string).

Eg: name = "Alex"
print(f"Hello, {name}!")

↓
curly braces.

→ This is for combining text.

→ to combine text + value (Numeric expression in an f-string):-

Eg: age = 25
print(f"Next year my age is {age + 1}")

Python string tools (tool box):- (methods = builtins) & use attach it with a dot (.)

- .upper() - for all caps
- .lower() - for lowercase
- .strip() - trims ends
- .replace() - swaps text
- len() - counts characters

String operations // to format & manipulate text.

they just give us a copy of data on console & doesn't change the values in the variables.

@ 15th - October - 2026 - Thursday :-

* In python we store information in variables.
* A variable is just a "labelled container" & that holds a value in memory & that you get to choose the label.
You could put a number in it, a piece of text, a decimal, or something else entirely.
And whenever you need that value later; you just call it by name.
In some languages you need to initially declare what type of data is that variable holding but, in python it figures it out automatically & use it at any time.

* Variable naming rules:

-> must start with a letter or underscore.
-> Letters, numbers, & underscores are only acceptable.
-> They are case-sensitive.

* Dynamic typing - that means you don't have to tell python on what type of variable is. Python figures it out automatically & reassign it a new value later. python quietly adjusts it later for you if necessary.

python always looks at the value at the right side first & then puts it into container or on the left.

name = "Yogesh" # all of this because of = sign.

sees the data type & later makes sure it's that variable named name is containing a data with type = text.

"right to left"

* at any point if want to see what kind of type the variable is holding, you can type the below syntax:

Syntax: print (type(age))
<class 'str'> # output

-> type() is a built in function in python.

* Modulo operator (an arithmetic operator) is mainly used to check whether the number is odd or even as it returns remainder value or left over value.

* Exponent operator (**): Two stars ** for powers.
Eg: 2 ** 3 = 8 (2x2x2)

* Python follows PEMDAS rule.

Eg: result1 = 2 + 3 * 4 # without parentheses
result2 = (2+3) * 4 # with parentheses
print (result1, result2)
output 14 20

Use parentheses if you actually want to run something in a sequence. especially while arithmetic operations.

* Mixing integers & floats: int + float = float
as decimal precision & hence python doesn't want to loose decimal wins.

-> preserves decimal precision -> Automatic conversion
Eg: result1 = 5+2.
result2 = 5 + 2.5.
print (type(result1), type(result2))
output 7 <class 'int'> 7.5 <class 'float'> -> automatic conversion

* Don't divide by zero:

dividing numbers by zero will cause your program to crash. As python doesn't allow it. If you try then python returns zero division error message.

// = double division or Integer division
** = Exponentiation

* Type casting:- As a default feature python always set the data type to text when a value is assigned by user from console & in order change it's type the concept of "type casting" comes into frame.

* Casting = converting from one type to another:-

-> Three built-in converters:-

int() for whole numbers Eg: age = int(input("Age?"))
float() for ~~whole~~ decimal numbers Eg: CGPA = float(input("CGPA"))
str() for text Eg: Name = str(input("Name"))

* Wrap or nest:

wrap = two steps; Eg: raw = input("age")
age = int(raw)

nest = one line; Eg age = int(input("age"))

input runs → int wraps → age stores 42

↓

right-to-left rule of python

* limits of conversion:-

-> Non-Numeric text can't be parsed; Eg: Name = int(input(""))
-> Python won't drop the decimal for you. Eg: GPA = int(input(""))

* Rule of thumb:

-> If a decimal is possible, use float().
-> float convert to int() later.
-> python rounds down (drops the decimal)
-> Eg: int(3.7) equals 3

@ 19th October-2026-Monday:-

* To add delay in Python:- It's done by import a library & by adding the statement of delay.

Eg: import time
time.sleep()

language Python VS code

#Python - Run time, that executes code. (interpreter) (code editor)
#virtual environments & packages with pip

* ctrl + shift + x = open extension tab.
* ctrl + shift + P = terminal (select default profile) to select git bash / terminal / shell.

☆ Managing packages with pip :-

☆ Virtual environment creation :-

python -m venv .venv
.venv\Scripts\activate
S = capital
} used to create a virtual environment.

pip install requests } install's package into current env.
↳ Package name can be specified.

pip list } shows everything installed

pip freeze > requirements.txt } saves your dependencies to a requirements file.

pip install -r requirements.txt } lets anyone recreate your exact env on their machine.

* ctrl ⊕ shift ⊕ P -> Select interpreter (to create & activate a Venv)

* What is a virtual environment :- A virtual environment is an isolated folder that contains it's own copy of python & any packages you install. Think of it as a clean workspace for each project -- packages you install in one environment don't affect another.

☆ Reading error messages :-

* Error type - what went wrong
* Line number - exactly where it happened
* Message - what python expected instead.

} most imp debugging skills of a programmer

ctrl ⊕ shift ⊕ I = to open chat view with copilot.